CSCI218: Foundations of Artificial Intelligence

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Lectures on learning

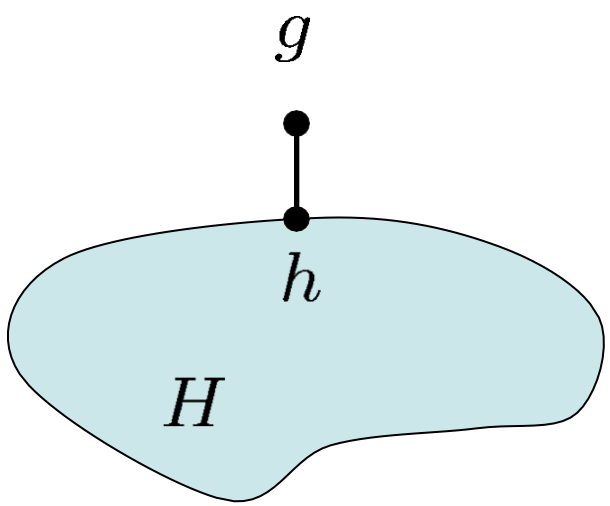
- ***Learning***: a process for improving the performance of an agent through experience
- Learning I (this):
  - The general idea: generalization from experience
  - Supervised learning: classification and regression
- Learning II: neural networks and deep learning

Supervised learning

- To learn an unknown ***target function*** f
- Input: a ***training set*** of ***labeled examples*** (x_j,y_j) where $y_j = f(x_j)$
  - E.g., x_j is an image, $f(x_j)$ is the label "giraffe"
  - E.g., x_j is a seismic signal, $f(x_j)$ is the label "explosion"
- Output: ***hypothesis*** h that is "close" to f, i.e., predicts well on unseen examples ("***test set***")
- Many possible hypothesis families for h
  - Linear models, logistic regression, neural networks, decision trees, examples (nearest-neighbor), grammars, kernelized separators, etc etc
- ***Classification*** = learning f with discrete output value
- ***Regression*** = learning f with real-valued output value

Inductive Learning (Science)

- Simplest form: learn a function from examples
  - A target function: g
  - Examples: input-output pairs $(x, g(x))$
  - E.g. x is an email and $g(x)$ is spam / ham
  - E.g. x is a house and $g(x)$ is its selling price
- Problem:
  - Given a hypothesis space H
  - Given a training set of examples x_i
  - Find a hypothesis $h(x)$ such that $h \sim g$
- Includes:
  - Classification (outputs = class labels)
  - Regression (outputs = real numbers)



Classification example: Object recognition

x

f(x) giraffe giraffe giraffe llama llama llama

X=

f(x)=?

Example: Spam Filter

- Input: an email
- Output: spam/ham

- Setup:
  - Get a large collection of example emails, each labeled “spam” or “ham” (by hand)
  - Learn to predict labels of new incoming emails
  - Classifiers reject 200 billion spam emails per day

- Features: The attributes used to make the ham / spam decision
  - Words: FREE!
  - Text Patterns: $dd, CAPS
  - Non-text: SenderInContacts, AnchorLinkMismatch
  - …

Dear Sir.

First, I must solicit your confidence in this transaction, this is by virture of its nature as being utterly confidencial and top secret. …

TO BE REMOVED FROM FUTURE MAILINGS, SIMPLY REPLY TO THIS MESSAGE AND PUT "REMOVE" IN THE SUBJECT.

99 MILLION EMAIL ADDRESSES
FOR ONLY $99

Ok, Iknow this is blatantly OT but I'm beginning to go insane. Had an old Dell Dimension XPS sitting in the corner and decided to put it to use, I know it was working pre being stuck in the corner, but when I plugged it in, hit the power nothing happened.

Example: Digit Recognition

- Input: images / pixel grids
- Output: a digit 0-9

- Setup:
  - MNIST data set of 60K collection hand-labeled images
    - Note: someone has to hand label all this data!
  - Want to learn to predict labels of new digit images

- Features: The attributes used to make the digit decision
  - Pixels: (6,8)=ON
  - Shape Patterns: NumComponents, AspectRatio, NumLoops
  - …

0

1

2

1

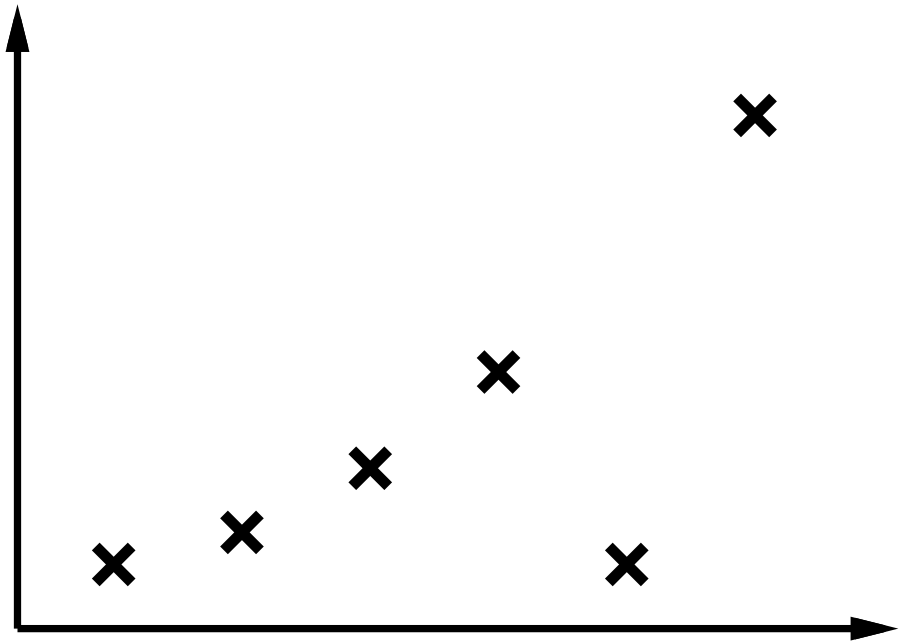
??

Other Classification Tasks

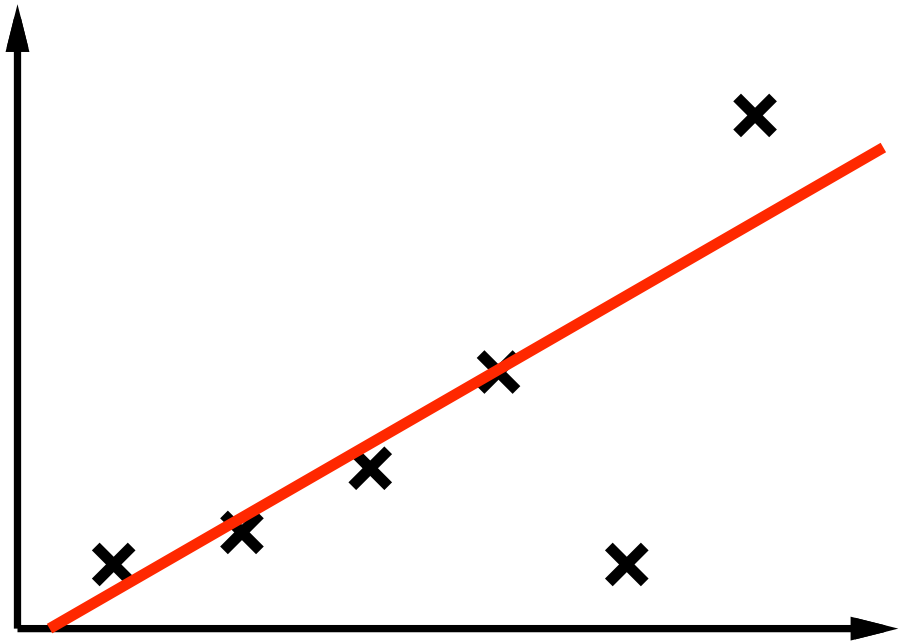
- Medical diagnosis
  - input: symptoms
  - output: disease
- Automatic essay grading
  - input: document
  - output: grades
- Fraud detection
  - input: account activity
  - output: fraud / no fraud
- Email routing
  - input: customer complaint email
  - output: which department needs to ignore this email
- Fruit and vegetable inspection
  - input: image (or gas analysis)
  - output: moldy or OK

- … many more

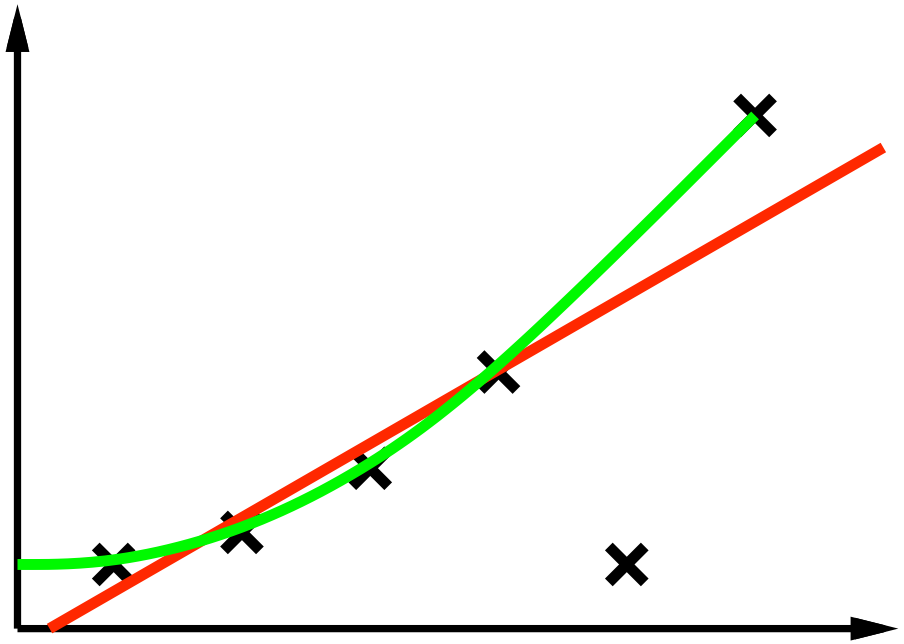
Regression example: Curve fitting



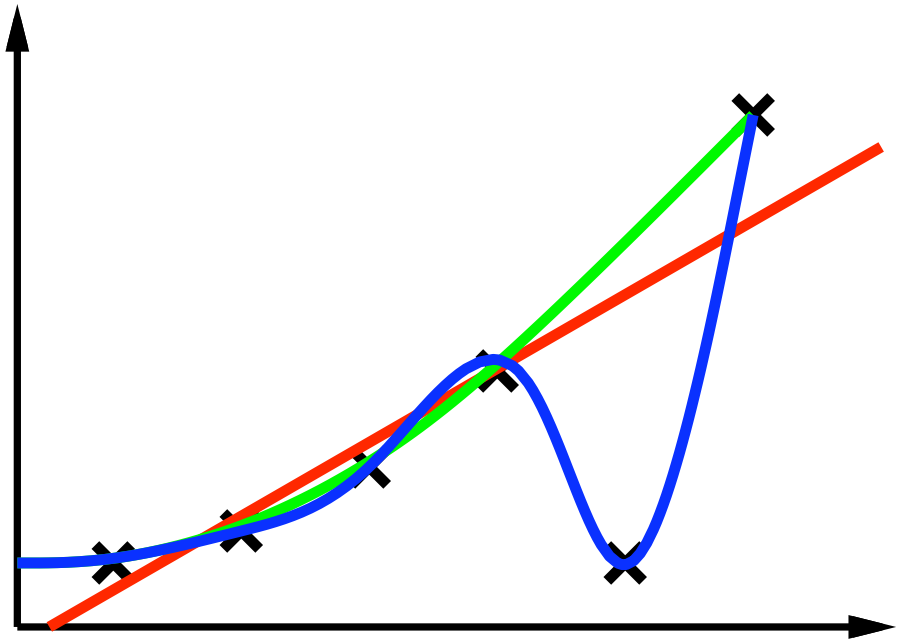
Regression example: Curve fitting



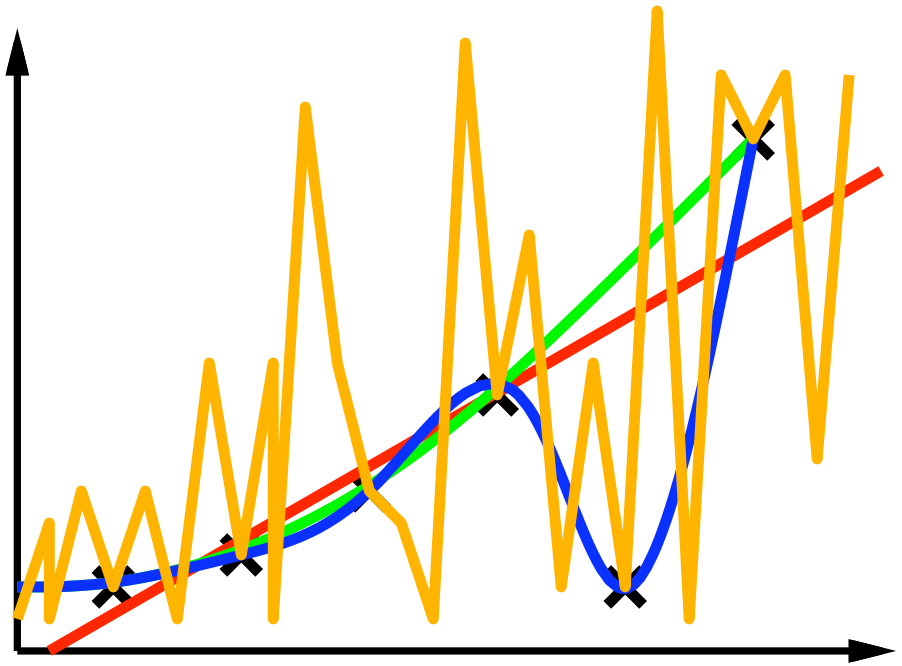
Regression example: Curve fitting



Regression example: Curve fitting



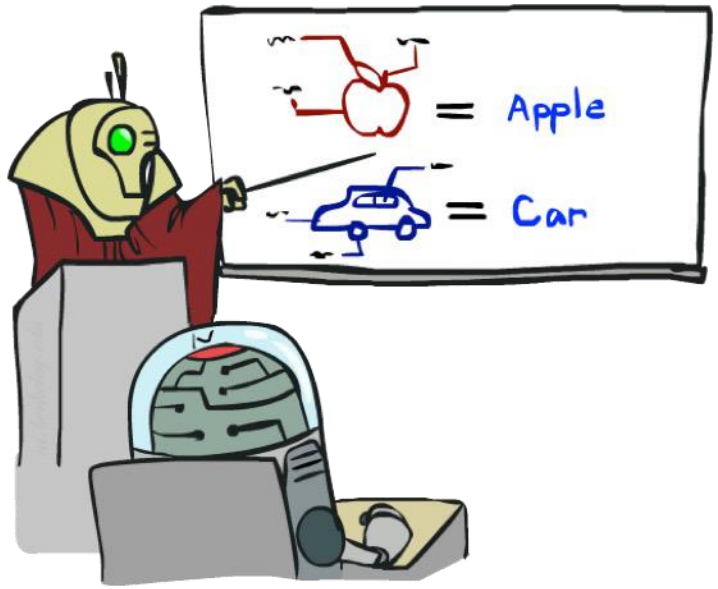
Regression example: Curve fitting

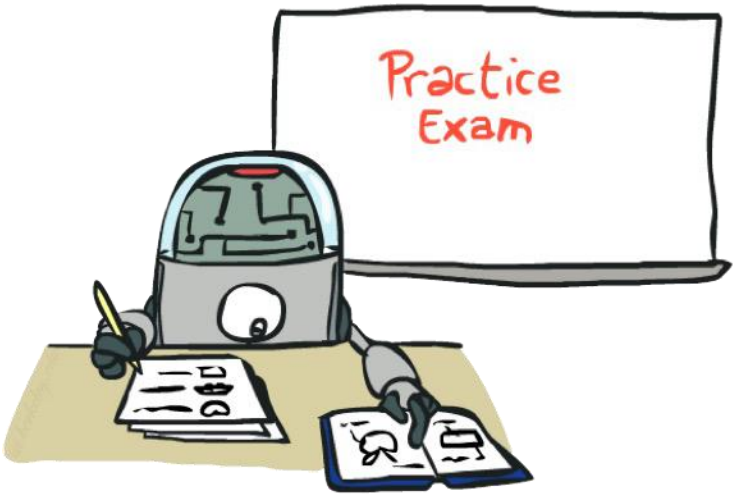


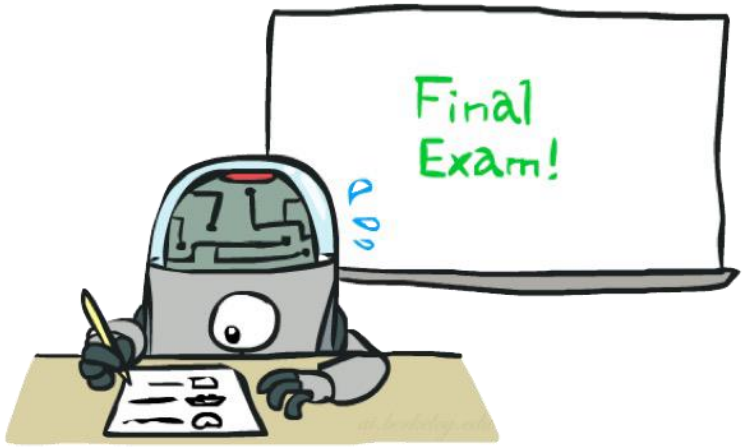
Basic questions

- Which hypothesis space H to choose?
- How to measure degree of fit?
- How to trade off degree of fit vs. complexity?
  - "*Ockham's razor*"
- How do we find a good h?
- How do we know if a good h will predict well?

Training and Testing







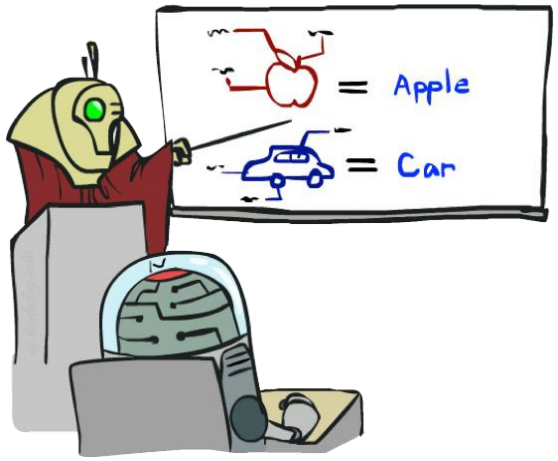
A few important points about learning

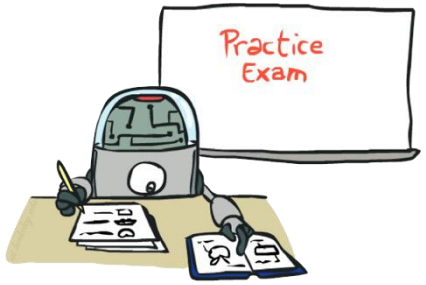
- Data: labeled instances, e.g. emails marked spam/ham
  - Training set
  - Held out set
  - Test set
- Features: attribute-value pairs which characterize each x
- Experimentation cycle
  - Learn parameters (e.g. model probabilities) on training set
  - (Tune hyperparameters on held-out set)
  - Compute accuracy of test set
  - Very important: never "peek" at the test set!
- Evaluation
  - Accuracy: fraction of instances predicted correctly
- Overfitting and generalization
  - Want a classifier which does well on *test* data
  - Overfitting: fitting the training data very closely, but not generalizing well
  - Underfitting: fits the training set poorly

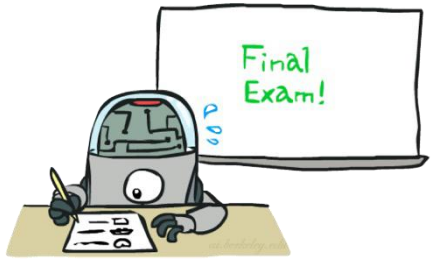
Training Data

Held-Out Data (Validation set)

Test Data

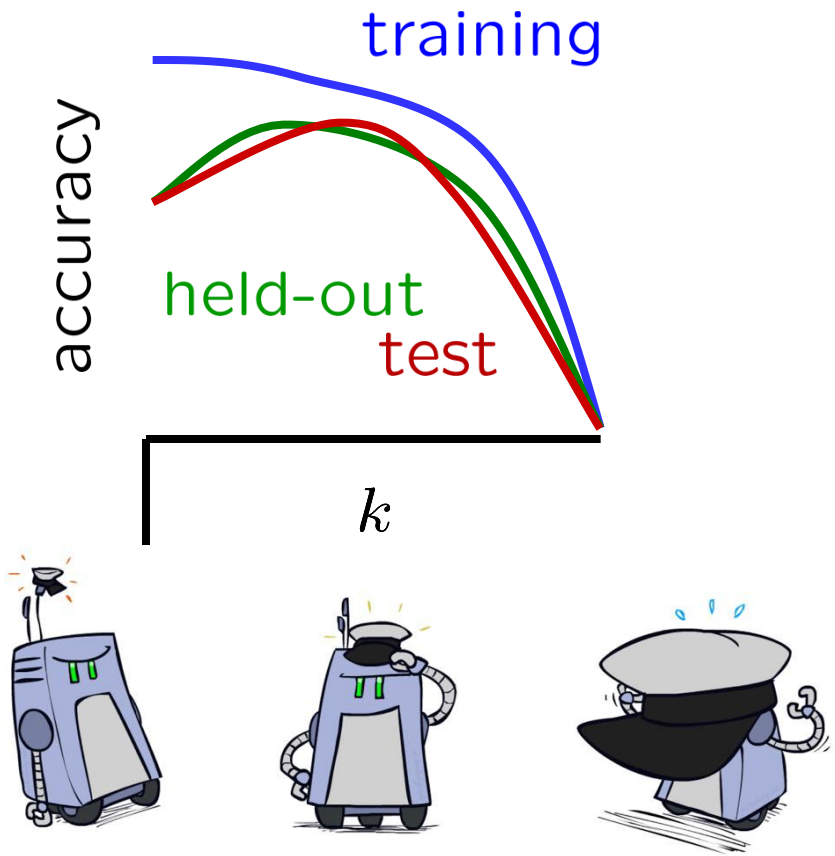






A few important points about learning

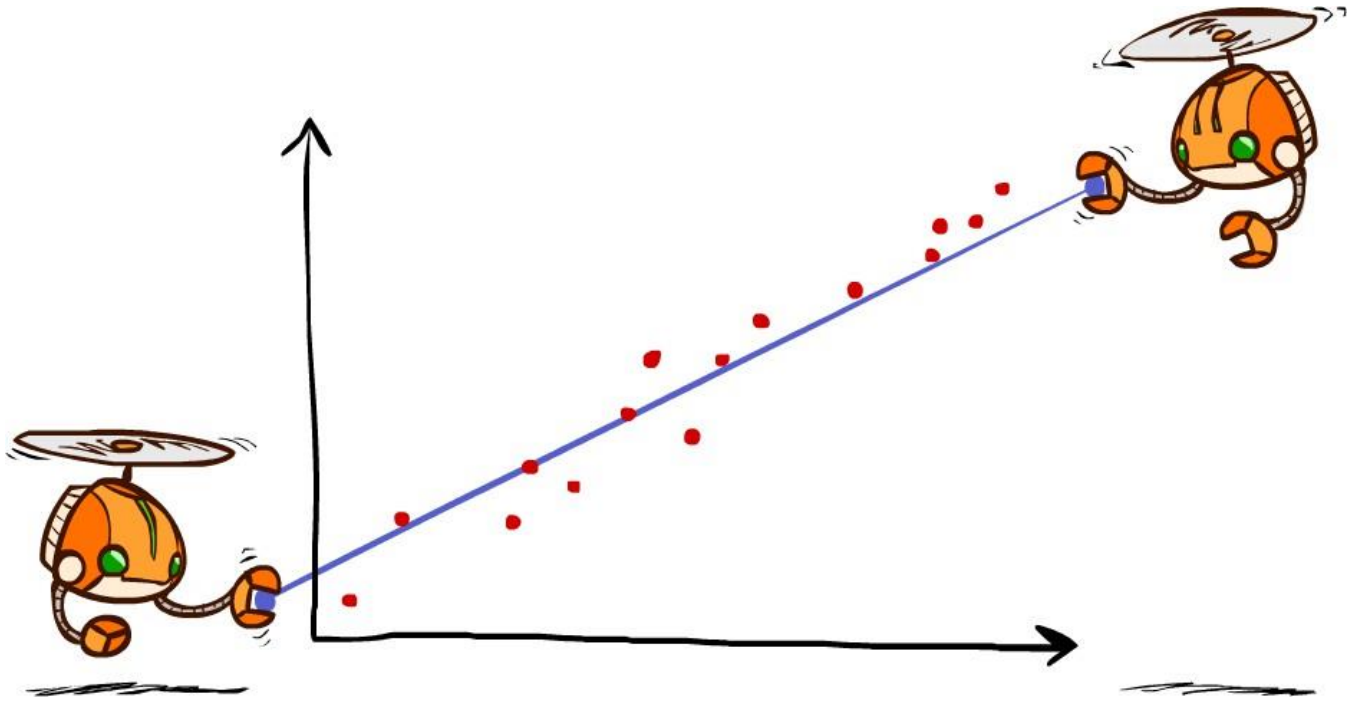
- What should we learn where?
  - Learn parameters from training data
  - Tune hyperparameters on different data
    - Why?
  - For each value of the hyperparameters, train and test on the held-out data
  - Choose the best value and do a final test on the test data
- What are examples of hyperparameters?



Supervised Learning

- ***Classification*** = learning f with discrete output value
- ***Regression*** = learning f with real-valued output value

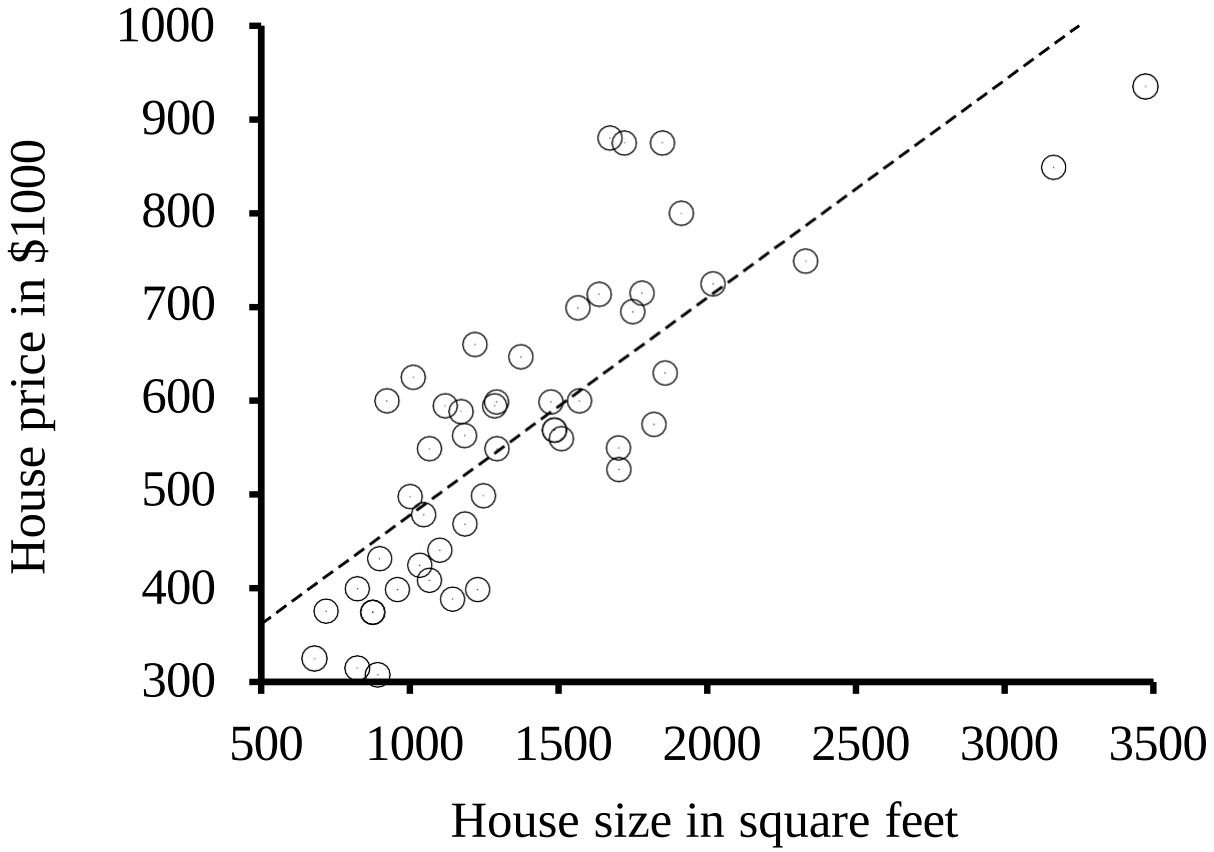
Linear Regression



Hypothesis family: Linear functions

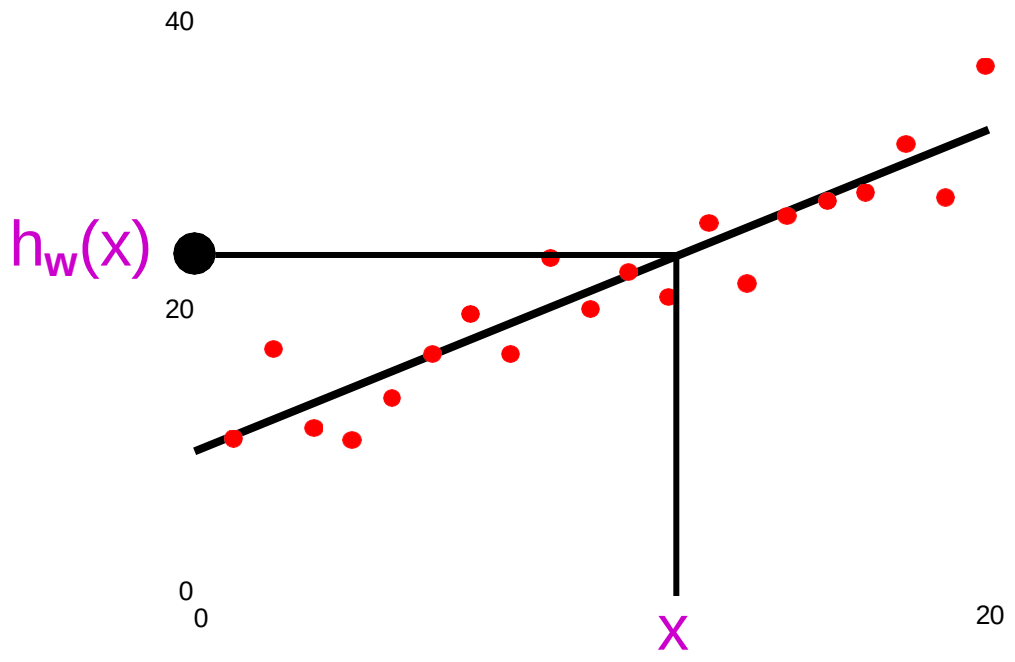
Linear Regression

(x, y=f(x)), x: house size, y: house price

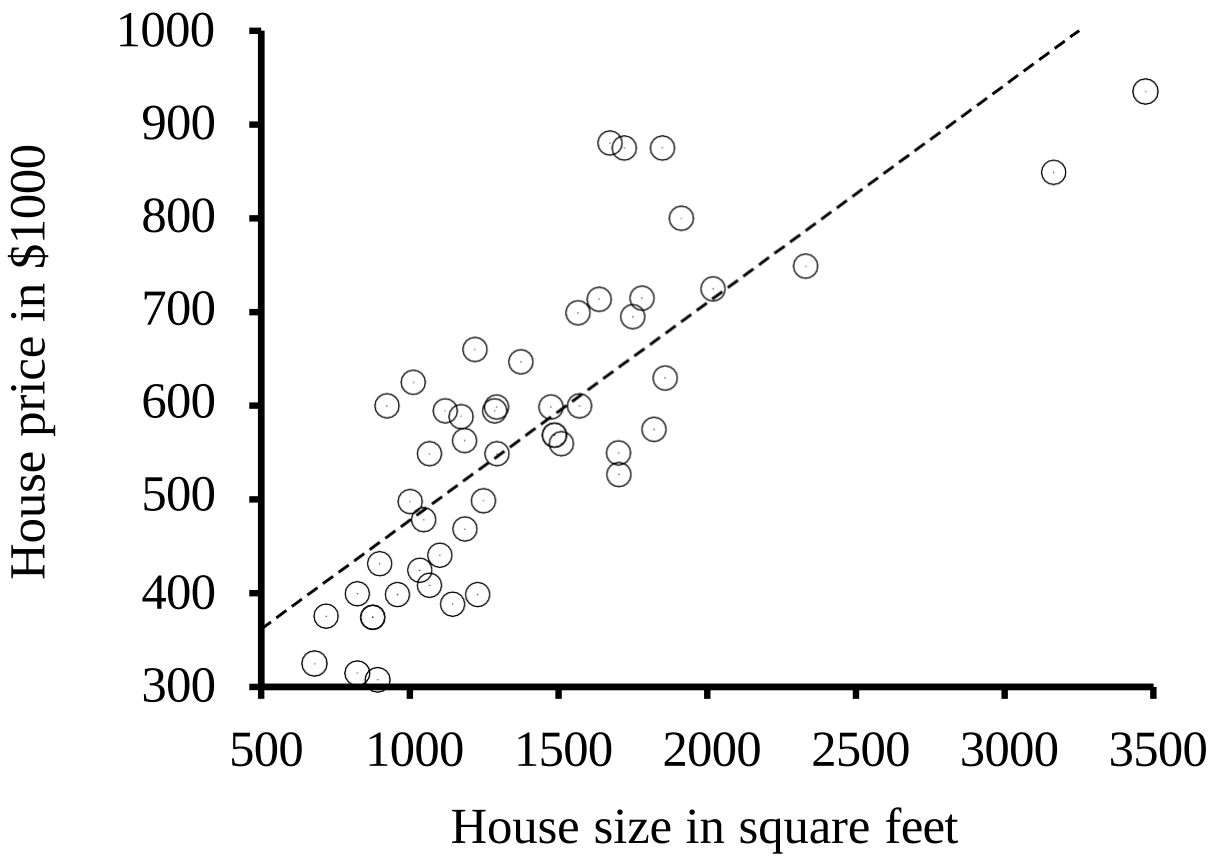


Berkeley house prices, 2009

Linear regression = fitting a straight line/hyperplane



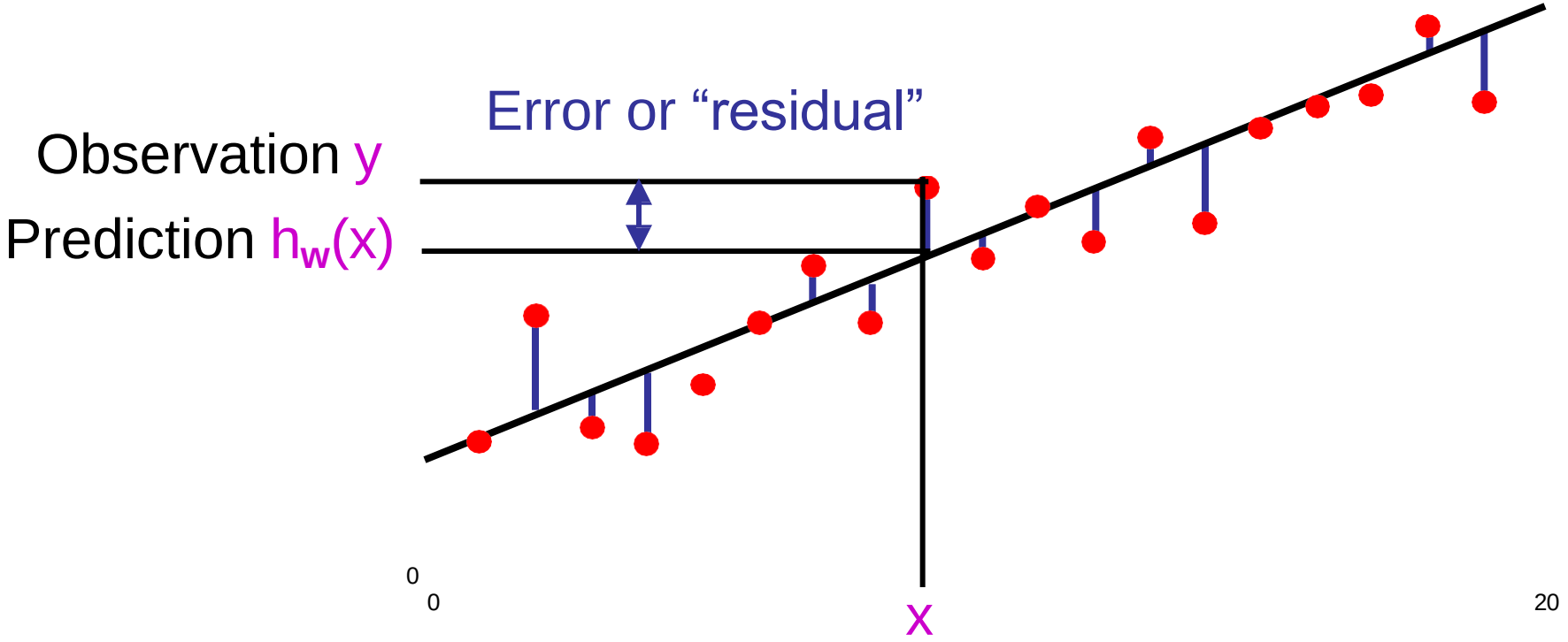
Prediction: $h_w(x) = w_0 + w_1 x$



Berkeley house prices, 2009

Prediction error

Error on one instance: $y - h_w(x)$



Find w

- Define loss function
- Find w* to minimize loss function

Least squares: Minimizing squared error

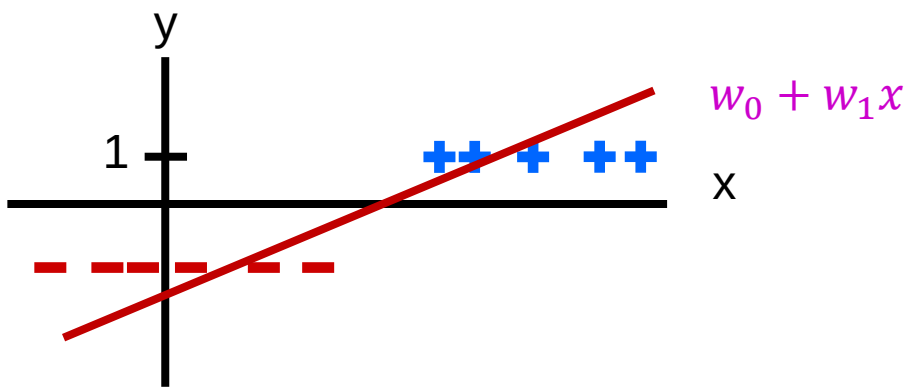
- L2 loss function: sum of squared errors over all examples
  - Loss = _____
- We want the weights w* that minimize loss
- At w* the derivatives of loss w.r.t. each weight are zero:
  - $\partial Loss/\partial w_0 =$ _____
  - $\partial Loss/\partial w_1 =$ _____
- Exact solutions for N examples:
  - $w_1 = [N\Sigma_j x_j y_j - (\Sigma_j x_j)(\Sigma_j y_j)]/[N\Sigma_j x_j{}^2 - (\Sigma_j x_j)^2]$ and $w_0 = 1/N[\Sigma_j y_j - w_1 \Sigma_j x_j]$
- For the general case where x is an n-dimensional vector
  - **X** is the data matrix (all the data, one example per row); **y** is the column of labels
  - $\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Least squares: Minimizing squared error

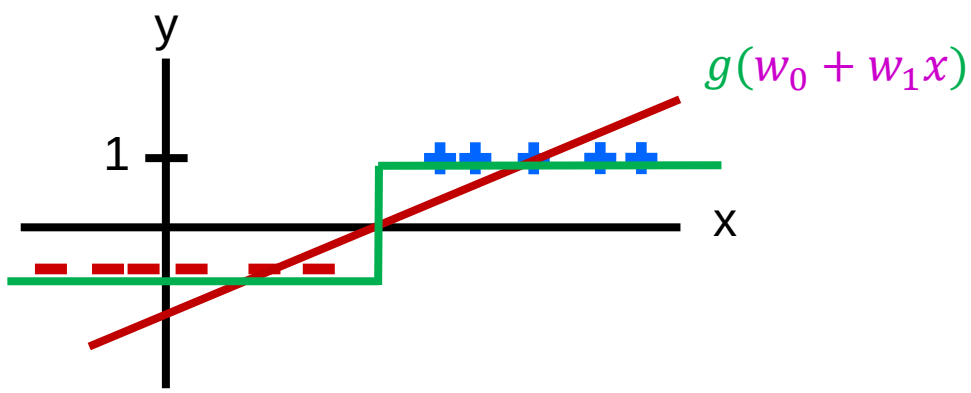
- L2 loss function: sum of squared errors over all examples
  - $Loss = \Sigma_j (y_j - h_{\mathbf{w}}(x_j))^2 = \Sigma_j (y_j - (w_0 + w_1 x_j))^2$
- We want the weights w* that minimize loss
- At w* the derivatives of loss w.r.t. each weight are zero:
  - $\partial Loss/\partial w_0 = -2\,\Sigma_j (y_j - (w_0 + w_1 x_j)) = 0$
  - $\partial Loss/\partial w_1 = -2\,\Sigma_j (y_j - (w_0 + w_1 x_j))\, x_j = 0$
- Exact solutions for N examples:
  - $w_1 = [N\Sigma_j x_j y_j - (\Sigma_j x_j)(\Sigma_j y_j)]/[N\Sigma_j x_j^2 - (\Sigma_j x_j)^2]$ and $w_0 = 1/N\,[\Sigma_j y_j - w_j x_j]$
- For the general case where x is an n-dimensional vector
  - **X** is the data matrix (all the data, one example per row); **y** is the column of labels
  - $\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Regression vs Classification

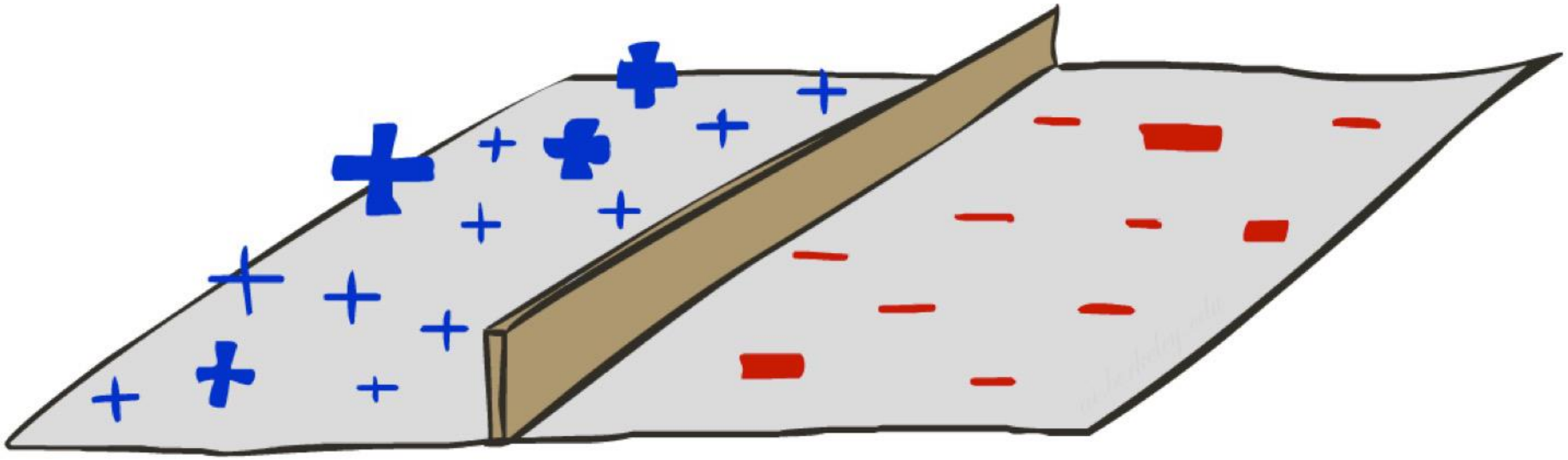
- Linear regression when output is binary, $y \in \{-1, 1\}$
  - $h_w(x) = w_0 + w_1 x$



- Linear classification
  - Used with discrete output values
  - Threshold a linear function
  - $h_w(x) = 1$ if $w_0 + w_1 x \geq 0$
  - $h_w(x) = -1$ if $w_0 + w_1 x < 0$
  - w: weight vector
  - Activation function g

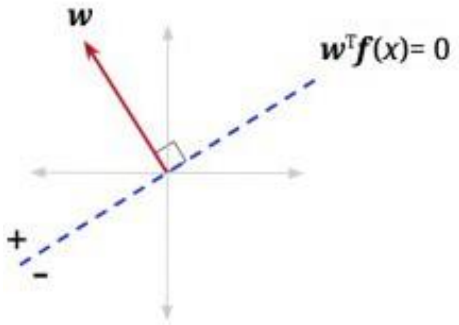


Threshold perceptron as linear classifier

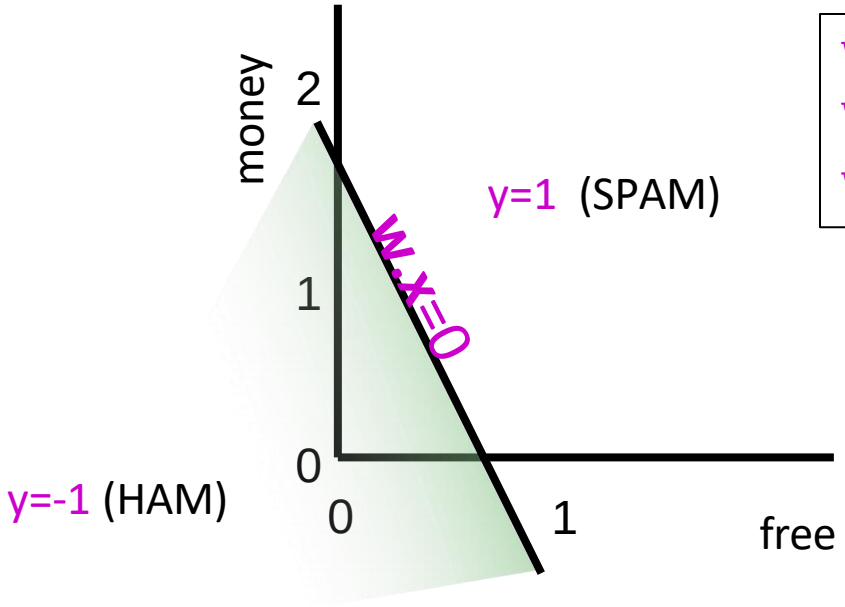


Binary Decision Rule

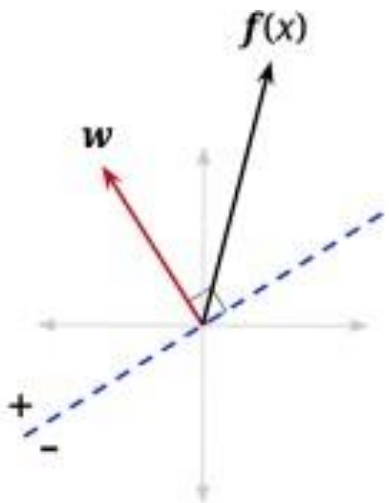
- A ***threshold perceptron*** is a single unit that outputs
  - $y = h_{\mathbf{w}}(\mathbf{x}) = 1$ when $\boldsymbol{w}.\boldsymbol{x} \geq 0$
    $= -1$ when $\boldsymbol{w}.\boldsymbol{x} < 0$
- In the input vector space
  - Examples are points **x**
  - The equation **w.x**=0 defines a ***hyperplane***
  - One side corresponds to y=1
  - Other corresponds to y=-1
  - Linear decision boundary



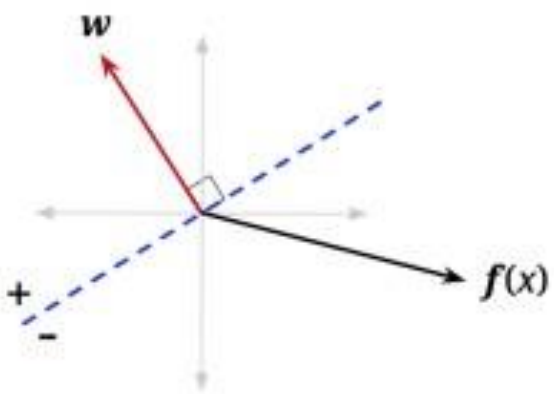
Decision Boundary



w_0	:	−3
w_{free}	:	4
w_{money}	:	2

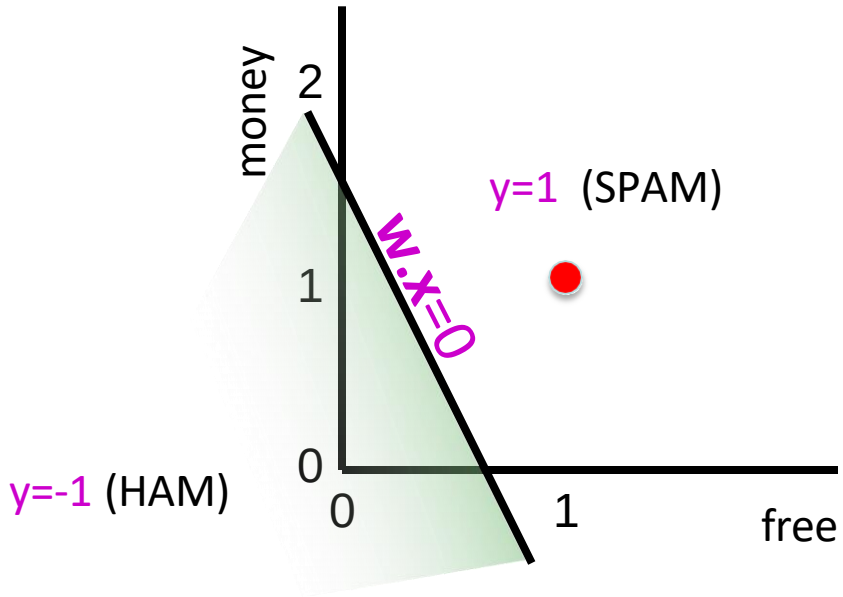


x classified into positive class



x classified into negative class

Example



w_0	:	-3
w_{free}	:	4
w_{money}	:	2

x_0	:	1
x_{free}	:	1
x_{money}	:	1

Dear Stuart, I'm leaving Macrosoft
to return to academia. The **money** is
is great here but I prefer to be **free**
to do my own research; and I *really*
love teaching undergrads!
Do I need to finish
my BA first before applying?
Best wishes
Bill

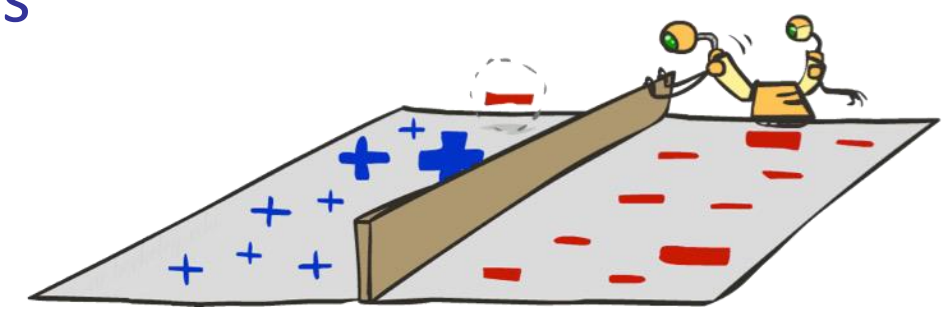
$$\mathbf{w.x} = -3x1 + 4x1 + 2x1 = 3$$

Weight Updates

Need a different solution than before given the characteristic of perceptron

Perceptron learning rule

- If true $y^* \neq h_{\boldsymbol{w}(\boldsymbol{x})}$ (an error), adjust the weights
- If **w.x** < 0 but the output should be y*=1
  - This is called a ***false negative***
  - Should ***increase*** weights on ***positive*** inputs
  - Should ***decrease*** weights on ***negative*** inputs
- If **w.x** > 0 but the output should be y*=-1
  - This is called a ***false positive***
  - Should ***decrease*** weights on ***positive*** inputs
  - Should ***increase*** weights on ***negative*** inputs



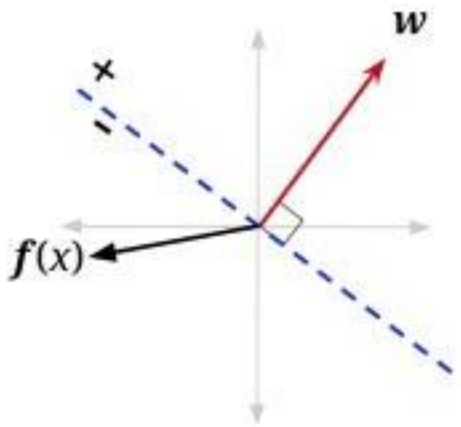
Perceptron Learning Rule

- Start with weights = 0
- For each training instance:
  - If wrong: adjust the weight vector by adding or subtracting the feature vector. y* is true label.

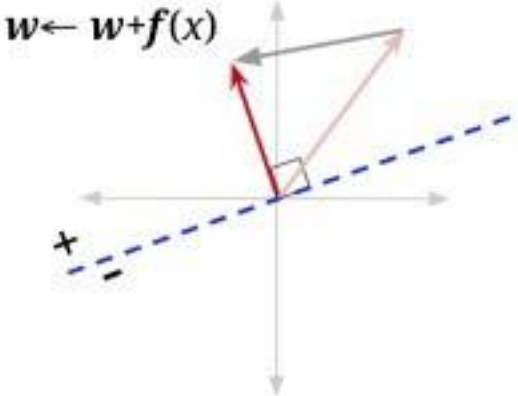
$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha\left(y^* - h_{\boldsymbol{w}}(\boldsymbol{x})\right)\boldsymbol{x}$$

$y = h_{\mathbf{w}}(\mathbf{x}) = 1$ when $\boldsymbol{w}.\boldsymbol{x} \geq 0$

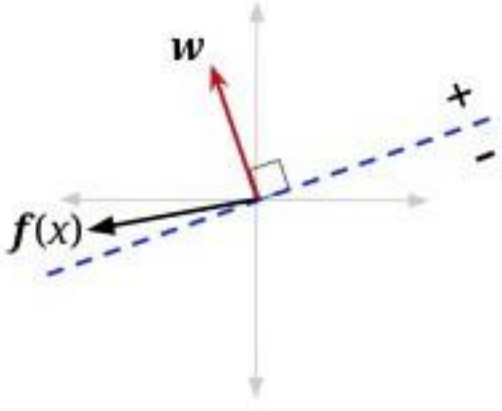
$= -1$ when $\boldsymbol{w}.\boldsymbol{x} < 0$



Mis-classifying x with old w

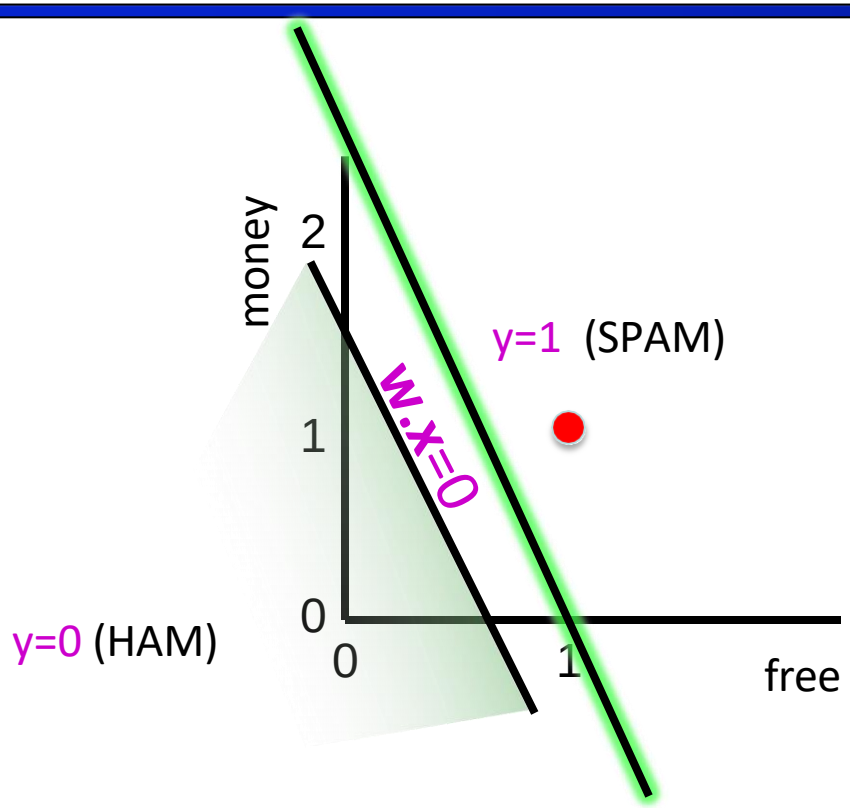


Updating w



Updated classification of x

Example



w_0	:	-3
w_{free}	:	4
w_{money}	:	2

x_0	:	1
x_{free}	:	1
x_{money}	:	1

Dear Stuart, I wanted to let you know that I have decided to leave Macrosoft and return to academia. The **money** is is great here but I prefer to be **free** to pursue more interesting research and I *really* love teaching undergraduates! Do I need to finish my BA first before applying?
Best wishes
Bill

w.x = -3x1 + 4x1 + 2x1 = 3

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha\,(y^* - h_w(x)\,\boldsymbol{x}$$

$\alpha = 0.5$

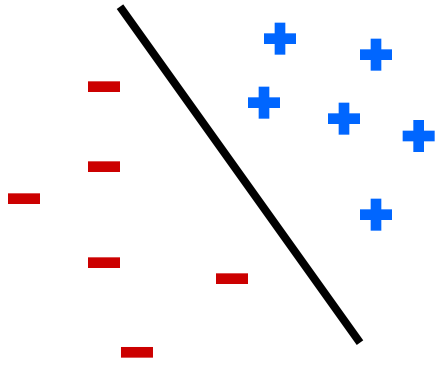
w ¬ (-3,4,2) + 0.5 (0 – 1) (1,1,1)
= (-3.5,3.5,1.5)

Perceptron convergence theorem

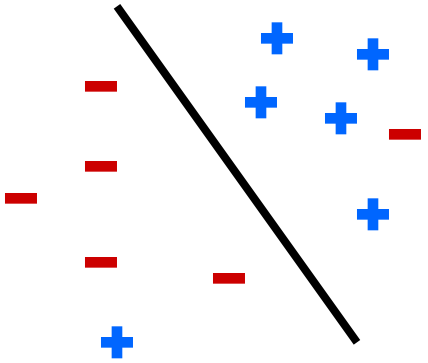
- A learning problem is ***linearly separable*** iff there is some hyperplane exactly separating positive from negative examples

- Convergence: if the training data are linearly ***separable***, perceptron learning applied repeatedly to the training set will eventually converge to a perfect separator

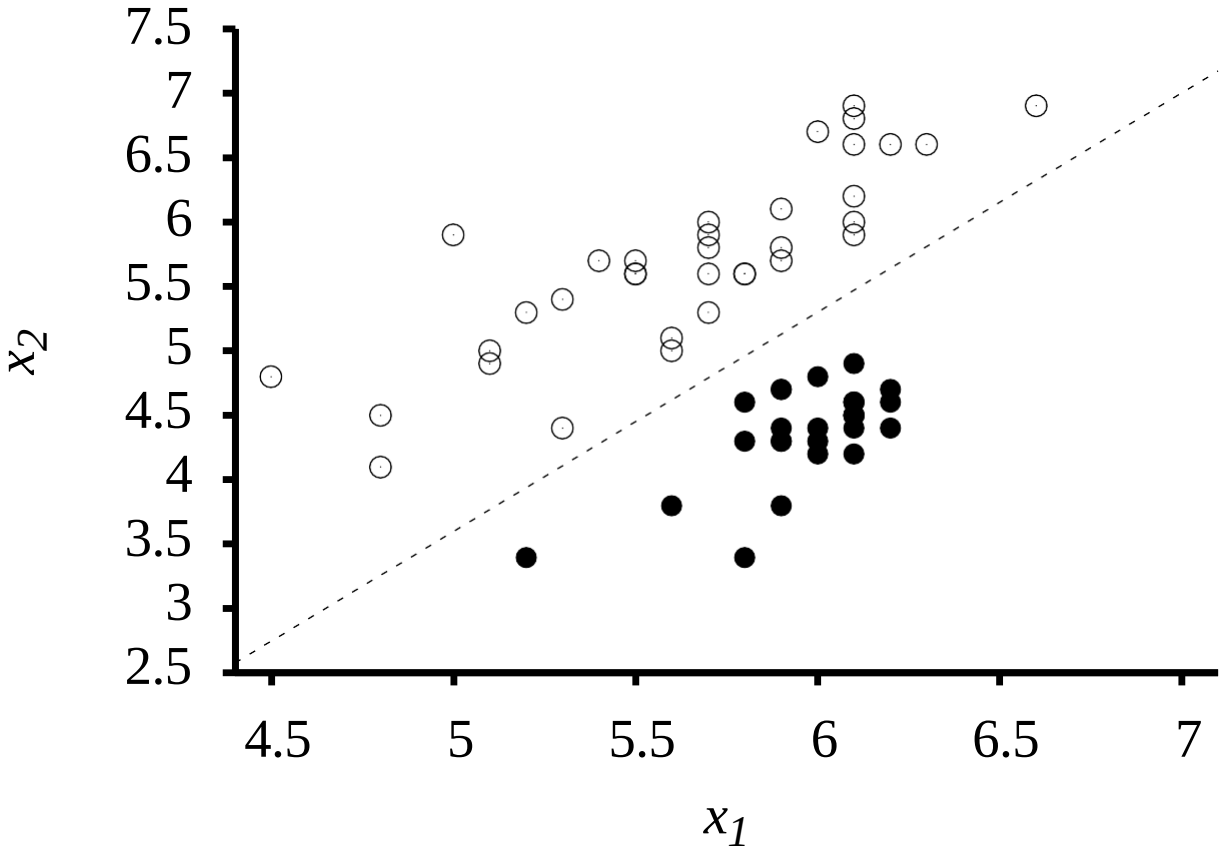
Separable

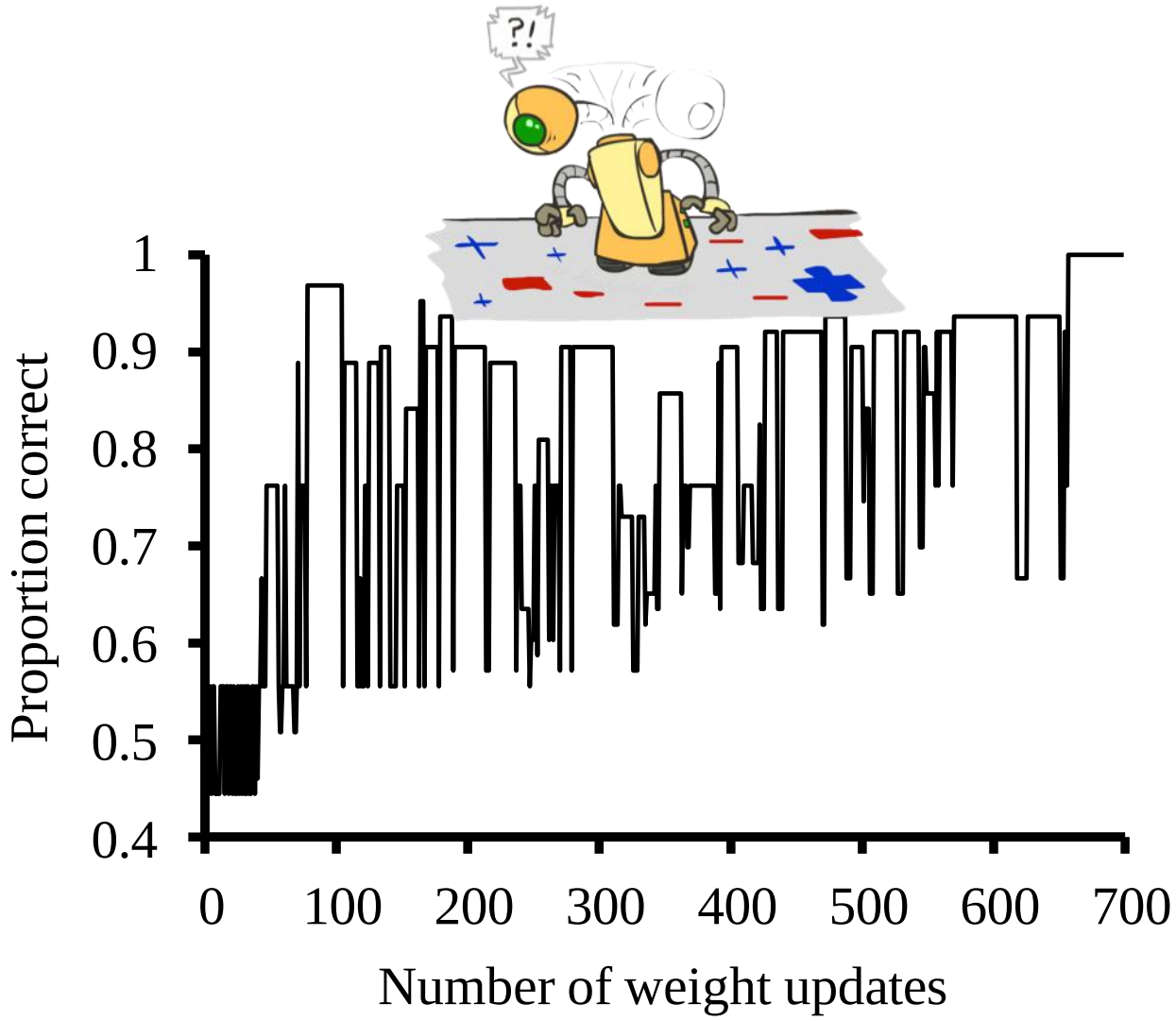


Non-Separable



Example: Earthquakes vs nuclear explosions





63 examples, 657 updates required

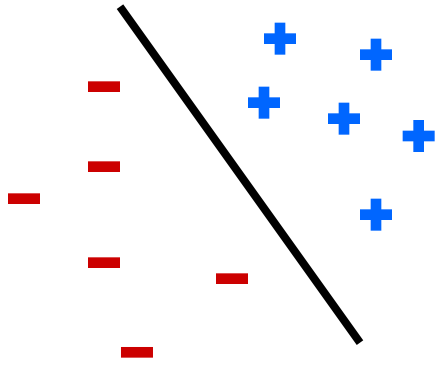
Perceptron convergence theorem

- A learning problem is ***linearly separable*** iff there is some hyperplane exactly separating +ve from –ve examples

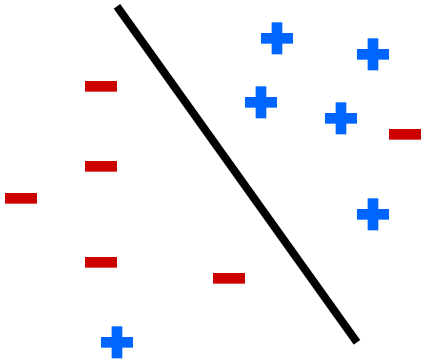
- Convergence: if the training data are separable, perceptron learning applied repeatedly to the training set will eventually converge to a perfect separator

- Convergence: if the training data are ***non-separable***, perceptron learning will converge to a minimum-error solution provided the learning rate α is decayed appropriately (e.g., $\alpha=1/t$)

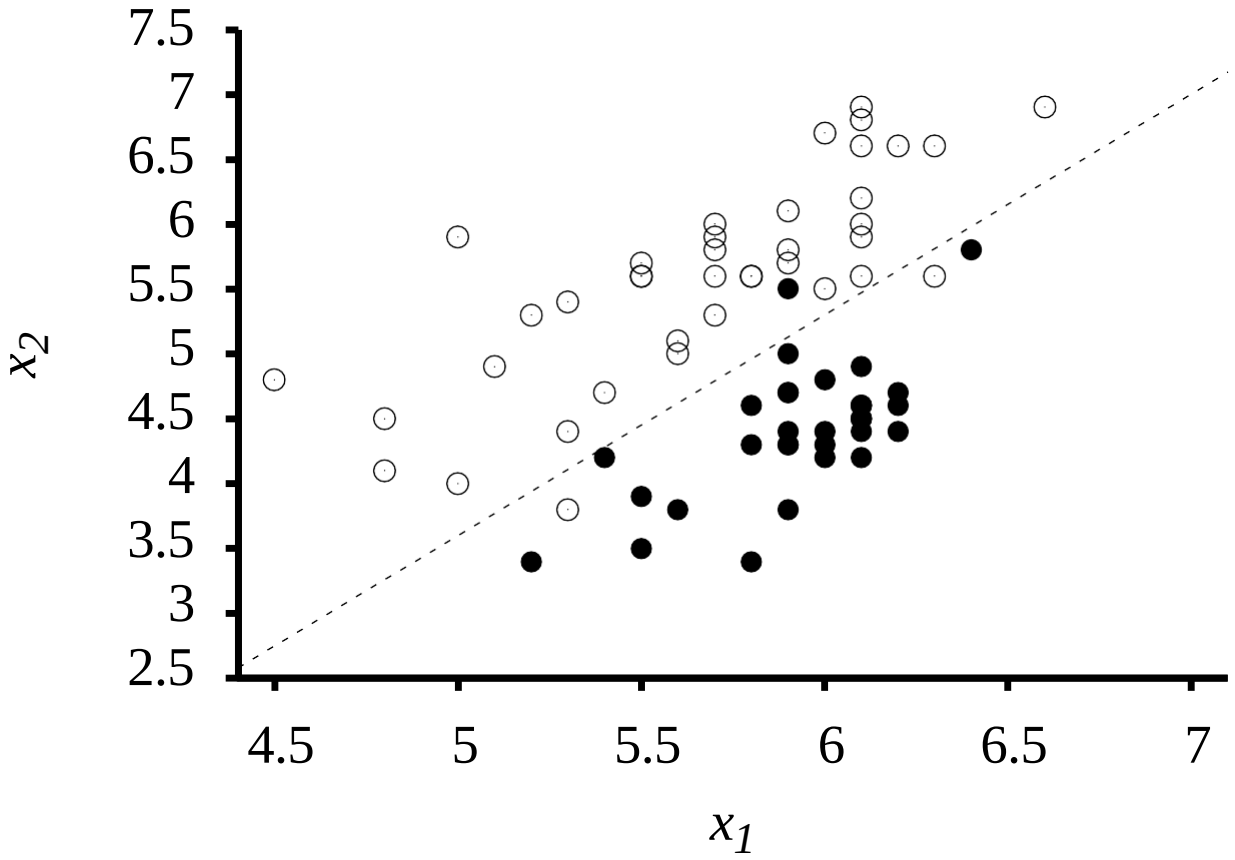
Separable

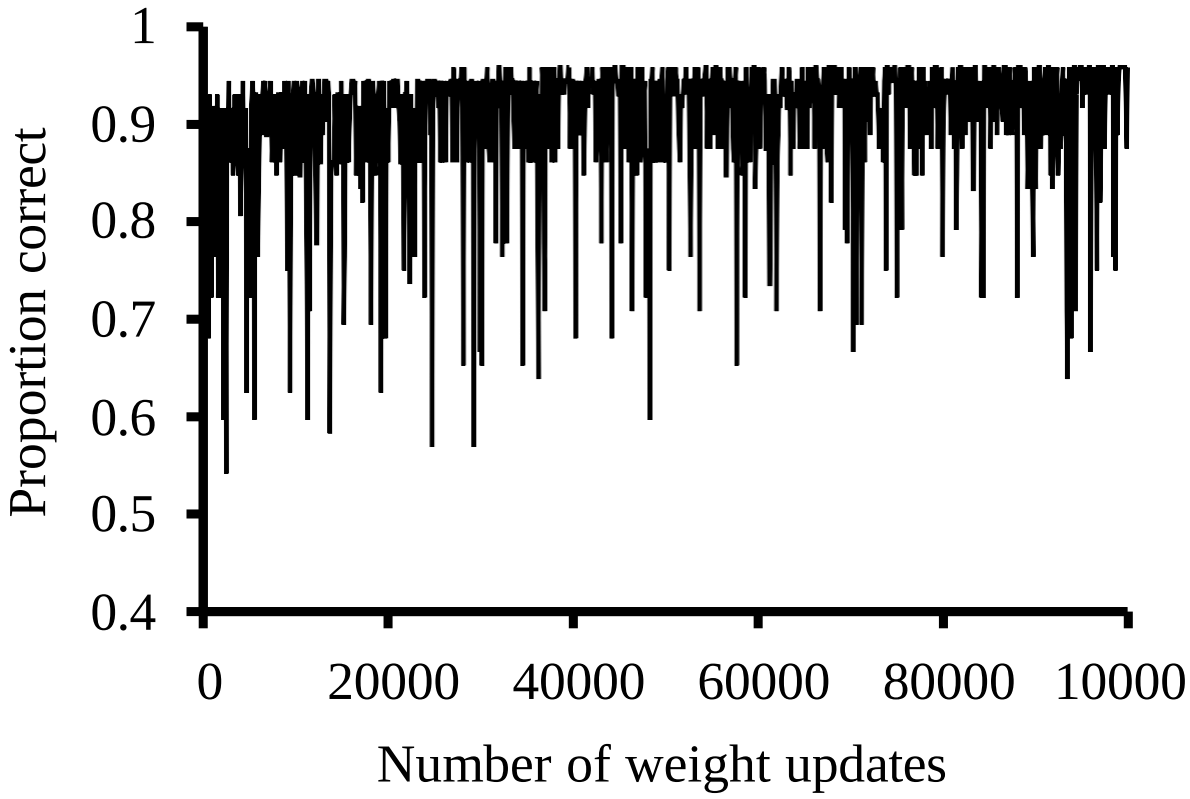


Non-Separable



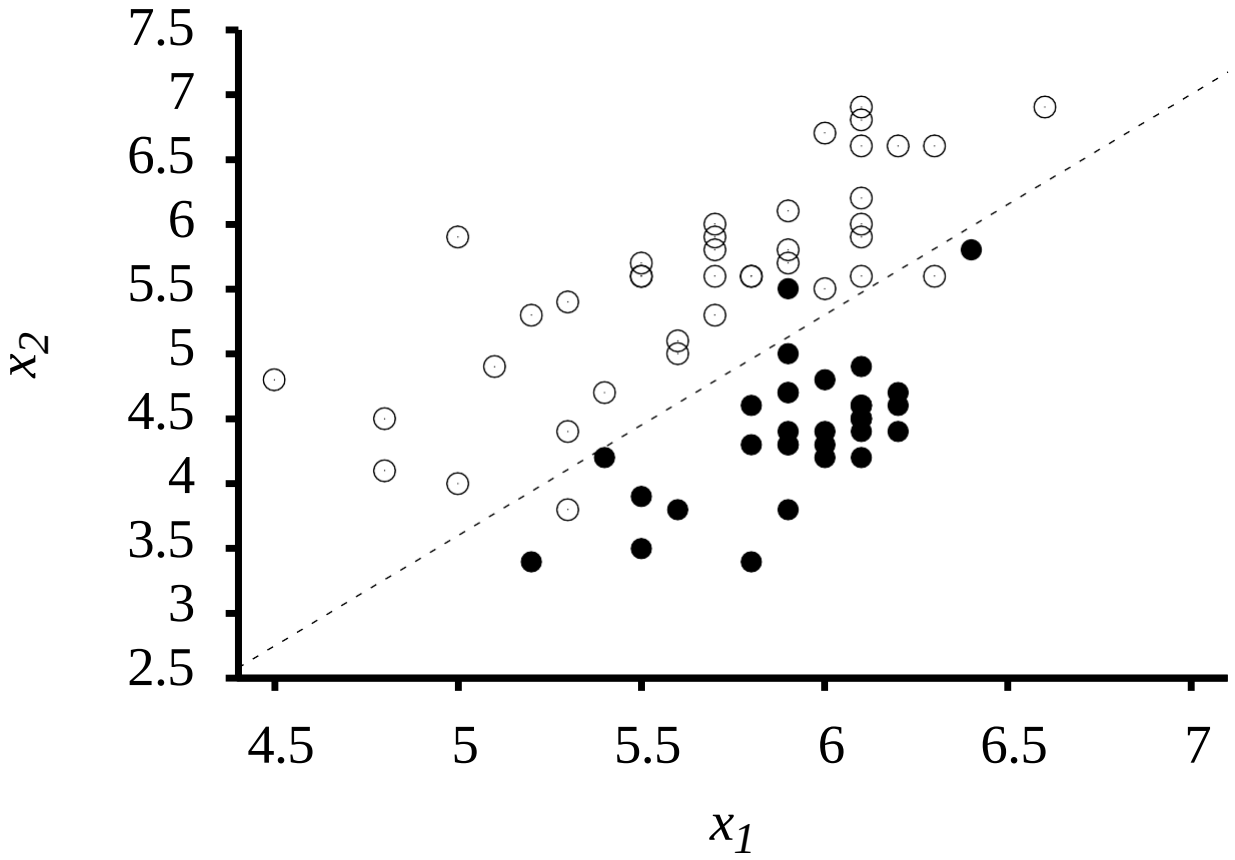
Perceptron learning with fixed α

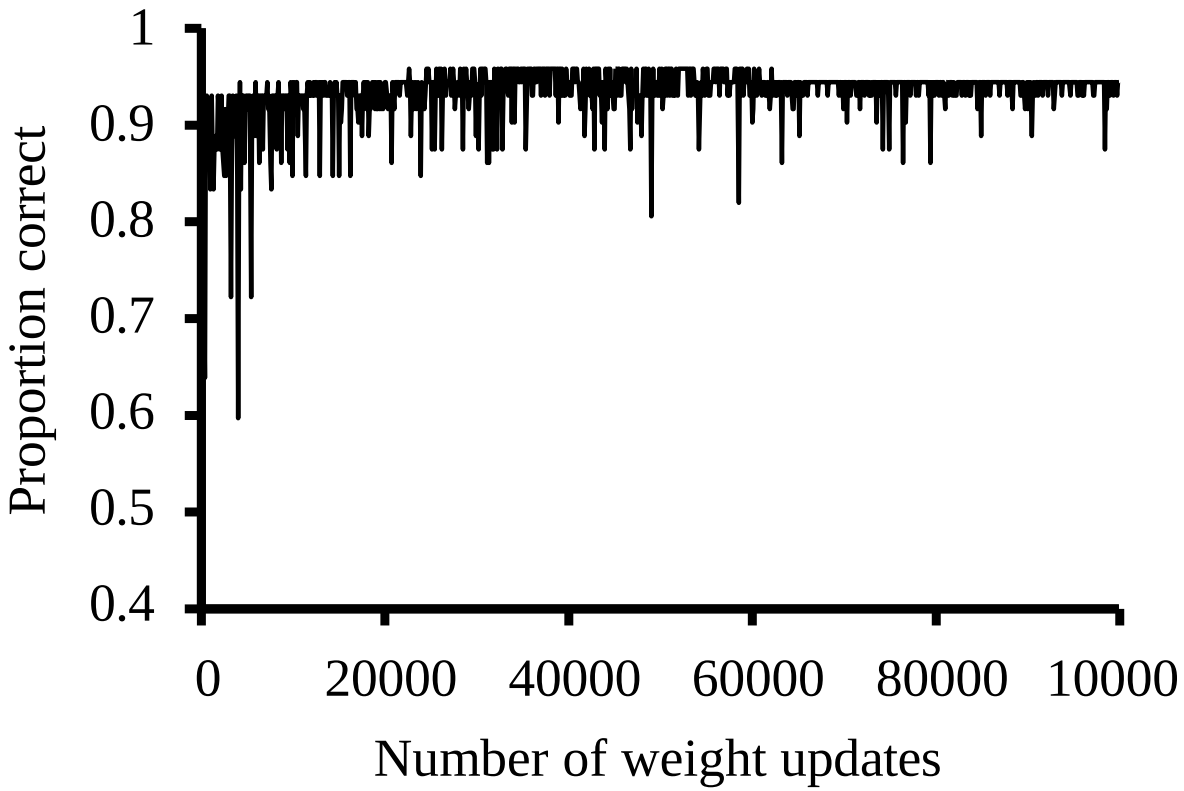




71 examples, 100,000 updates
fixed $\alpha = 0.2$, no convergence

Perceptron learning with decaying α

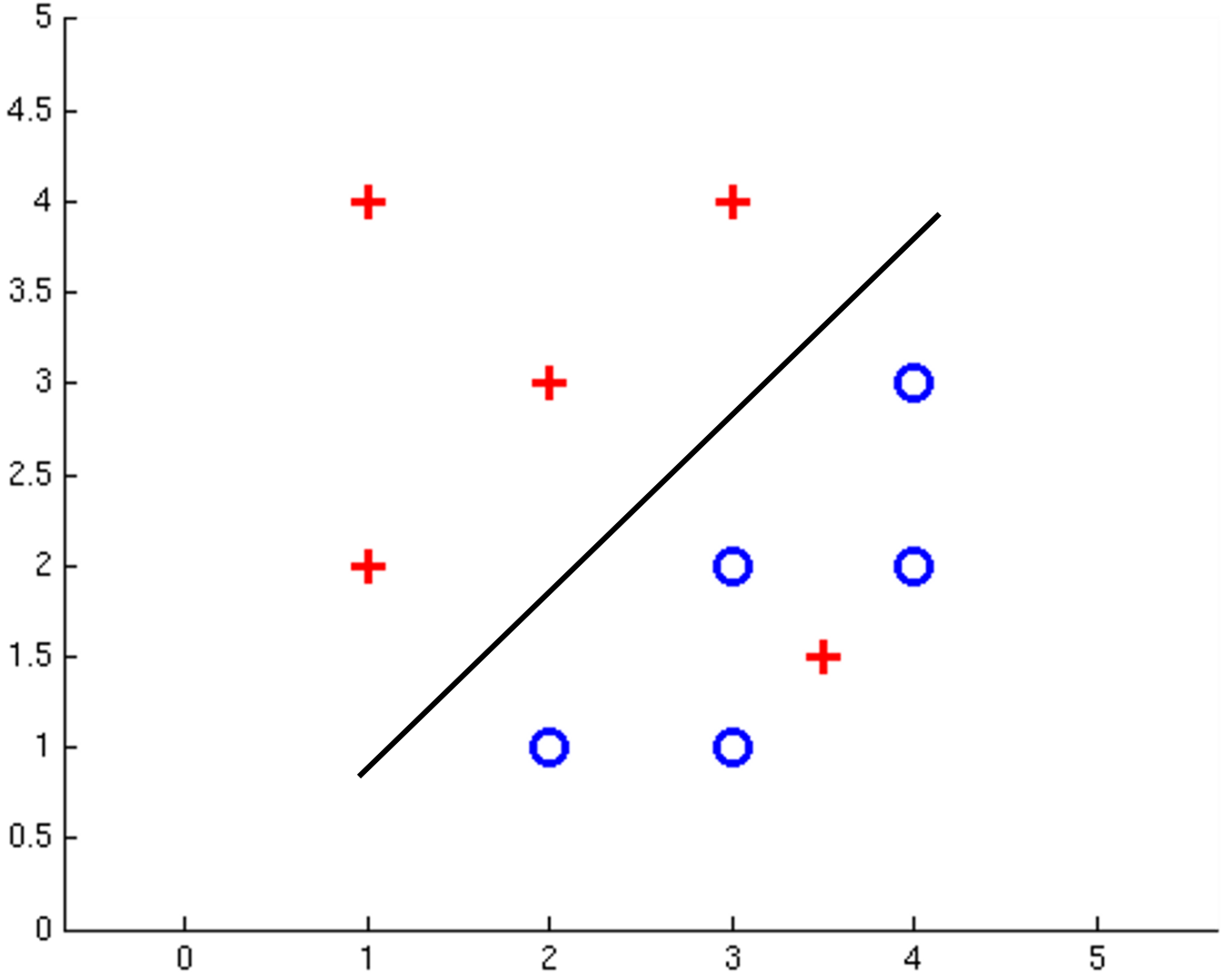




71 examples, 100,000 updates
decaying $\alpha = 1000/(1000 + t)$, near-convergence

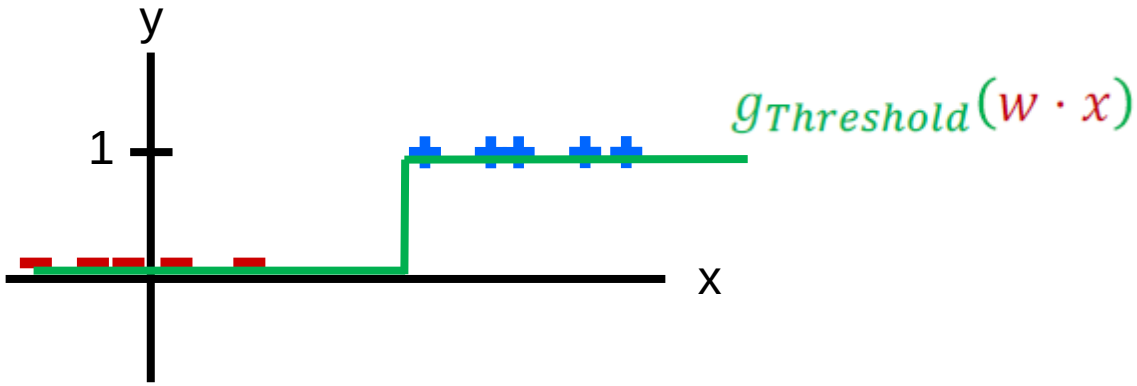
Non-Separable Case

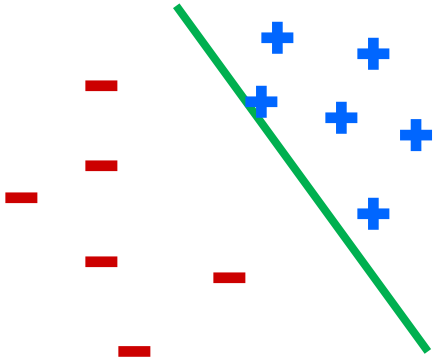
Even the best linear boundary makes at least one mistake

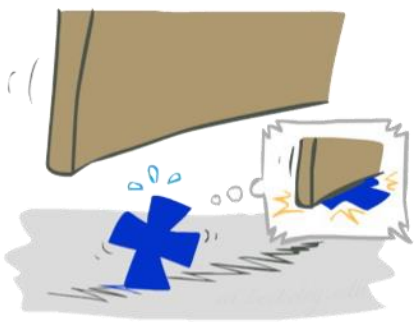


Other Linear Classifiers

- Perceptron is perfectly happy as long as it separates the training data

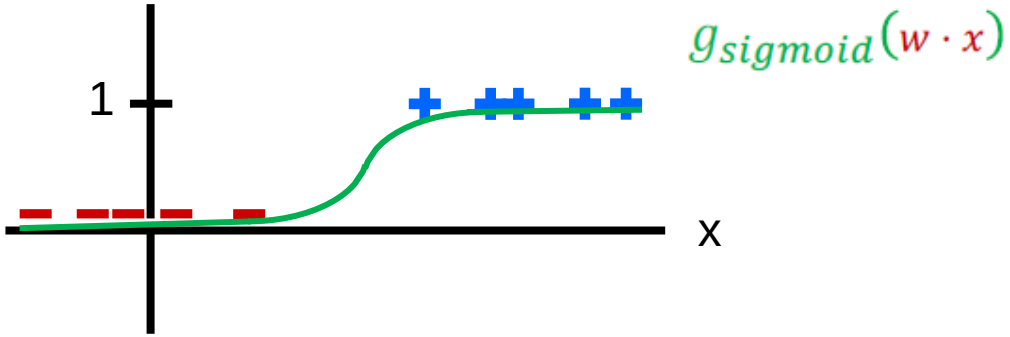




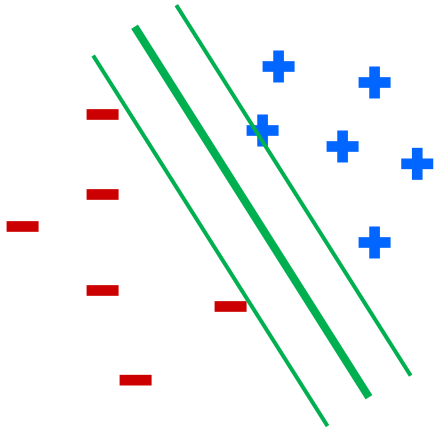


- Logistic Regression

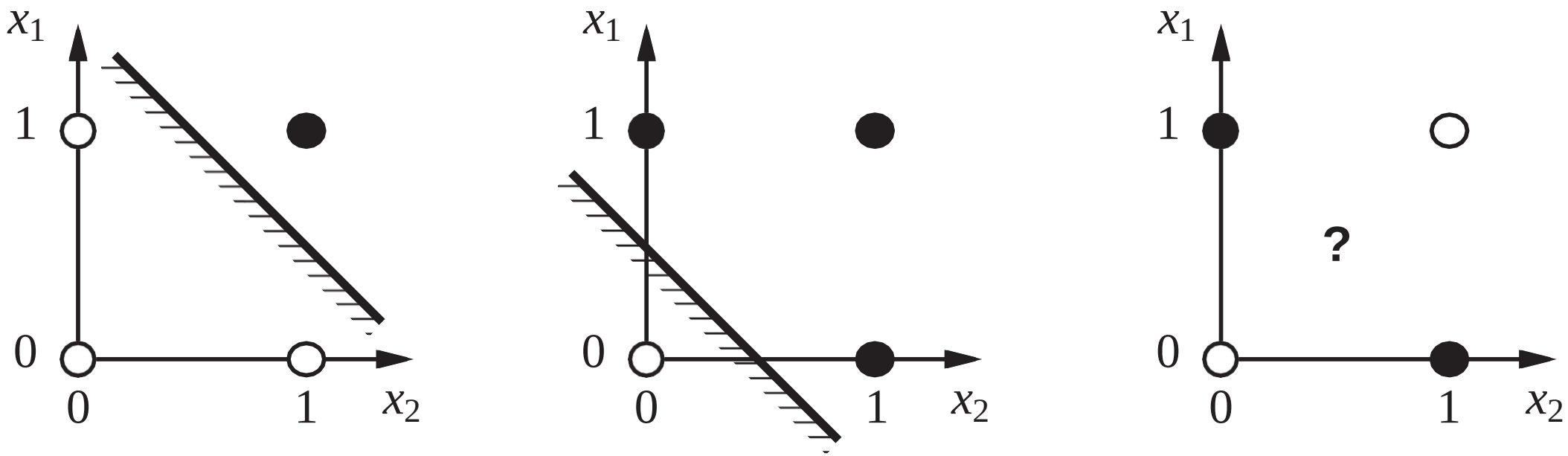
$$g_{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



- Support Vector Machines (SVM)
  - Maximize margin between boundary and nearest points



Perceptrons hopeless for XOR function



(a) x_1 **and** x_2 (b) x_1 **or** x_2 (c) x_1 **xor** x_2

Basic questions

- Which hypothesis space H to choose?
- How to measure degree of fit?
- How to trade off degree of fit vs. complexity?
  - "*Ockham's razor*"
- How do we find a good h?
- How do we know if a good h will predict well?

Classical stats/ML: Minimize loss function

- Which hypothesis space H to choose?
  - *E.g., linear combinations of features*: $h_w(x) = w^T x$
- How to measure degree of fit?
  - *Loss function, e.g., squared error* $\Sigma_j (y_j - w^T x)^2$
- How to trade off degree of fit vs. complexity?
  - *Regularization: complexity penalty, e.g.,* $||w||^2$
- How do we find a good h?
  - *Optimization (closed-form, numerical); discrete search*
- How do we know if a good h will predict well?
  - *Try it and see (cross-validation, bootstrap, etc.)*

Probabilistic: Max. likelihood, max. a priori

- Which hypothesis space H to choose?
  - *Probability model* $P(y \mid x,h)$, e.g., $Y \sim N(w^T x,\sigma^2)$
- How to measure degree of fit?
  - *Data likelihood* $\Pi_j P(y_j \mid x_j,h)$
- How to trade off degree of fit vs. complexity?
  - *Regularization or prior*: $\text{argmax}_h P(h) \Pi_j P(y_j \mid x_j,h)$ (*Max a priori*)
- How do we find a good h?
  - *Optimization (closed-form, numerical); discrete search*
- How do we know if a good h will predict well?
  - *Empirical process theory (generalizes Chebyshev, CLT, PAC…)*;
  - *Key assumption is (i)id*

Bayesian: Computing posterior over H

- Which hypothesis space H to choose?
  - *All hypotheses with nonzero a priori probability*
- How to measure degree of fit?
  - *Data probability, as for MLE/MAP*
- How to trade off degree of fit vs. complexity?
  - *Use prior, as for MAP*
- How do we find a good h?
  - ***<u>Don't!</u>*** *Bayes predictor* $P(y|x,D) = \Sigma_h P(y|x,h) P(D|h) P(h)$
- How do we know if a good h will predict well?
  - ***Silly question! Bayesian prediction is optimal!!***

Acknowledgement

The lecture slides are based on the materials from ai.Berkey.edu

Questions

UNIVERSITY
OF WOLLONGONG
AUSTRALIA